

Context Engineering: Memory, Windowing and Retrieval

Topics

- e! What is context engineering and why it matters for apps and agents
- e! Context windows: limits, costs, and trade-offs
- e! Context management strategies: chunking, summarisation, and windowing
- e! Conversational memory: buffer, summary, and vector memory
- e! Retrieval as context: grounding LLM responses
- e! Managing long documents and multi-turn conversation history
- e! Measuring and optimising token usage

Learning Objectives

By the end of this module, you will be able to:

- e! Explain the role of context engineering in AI applications and agents.
- e! Describe context-window limits, costs, and operational trade-offs.
- e! Apply chunking, summarisation, and windowing strategies.
- e! Compare buffer, summary, and vector memory approaches.
- e! Use retrieval to ground LLM responses in relevant information.
- e! Manage long documents and multi-turn conversation history.
- e! Measure and optimise token usage.

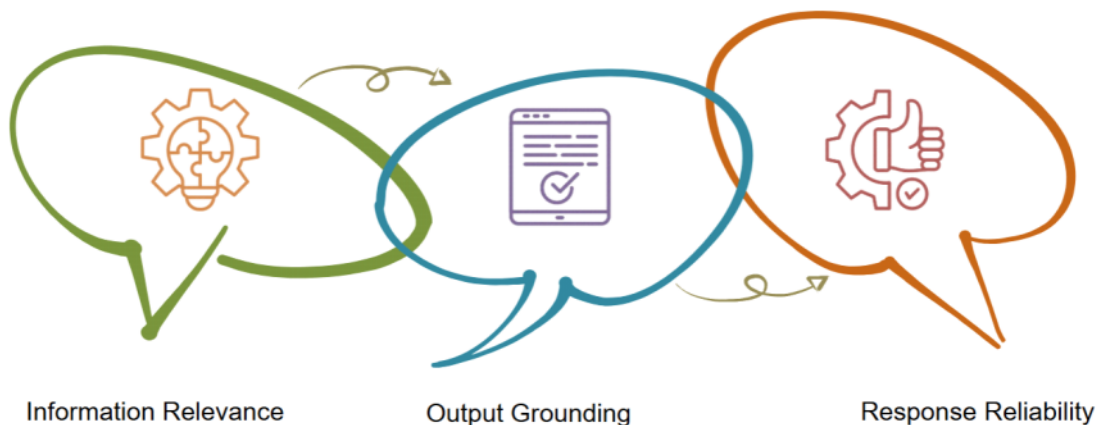
Introduction to Context Engineering

Introduction to Context Engineering

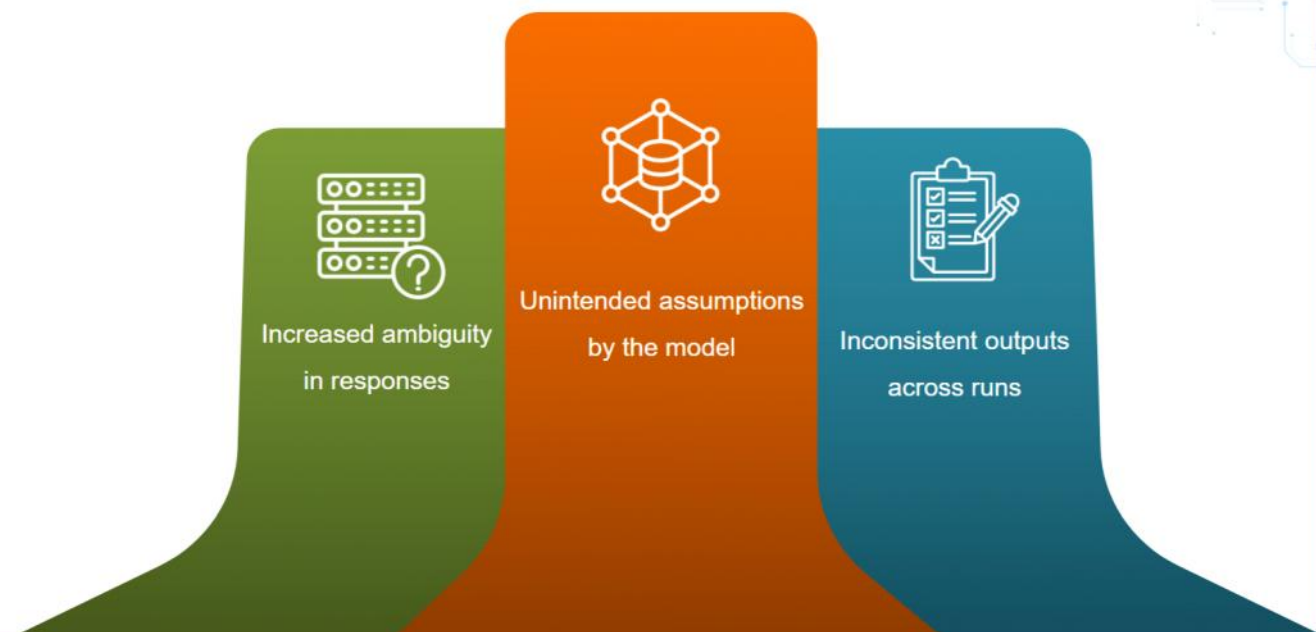
Context engineering is the practice of deliberately selecting, structuring, and positioning information to guide AI responses toward intended outcomes.

Introduction to Context Engineering

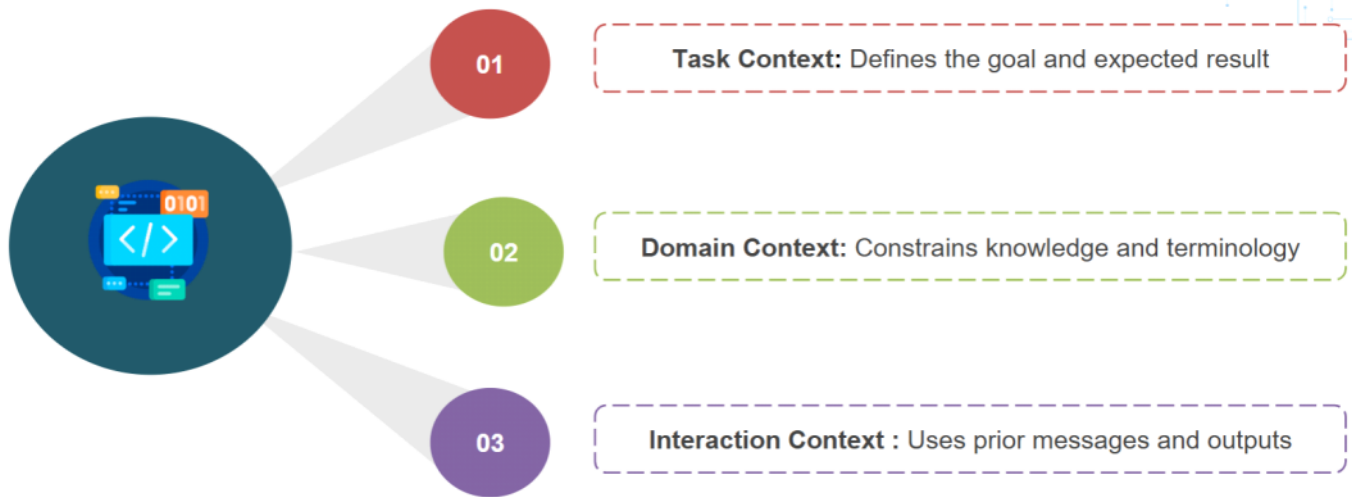
Context engineering is the practice of deliberately selecting, structuring, and positioning information to guide AI responses toward intended outcomes.



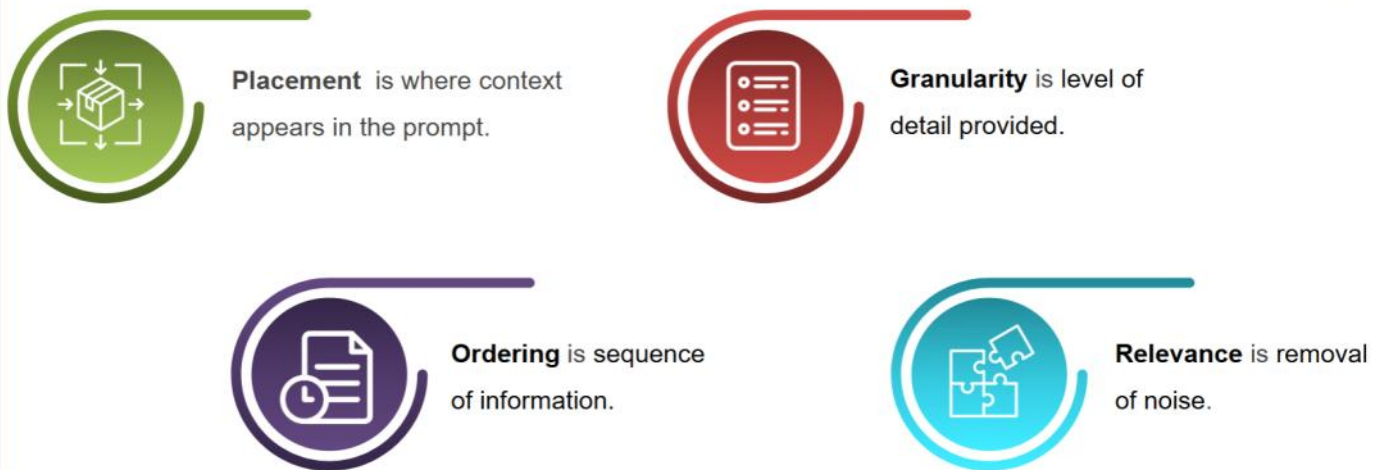
Impact of Missing Context



Core Context Dimensions



Structuring Context for AI



Benefits of Effective Context



Context Windows: Limits, Costs and Trade-Offs

Context Window: One Shared Token Budget

A single fixed budget holds **input and output together**, not two separate limits. Overflow is **Truncated**.

Context window — fixed size (e.g. 128k tokens)



Input tokens

system + history + prompt

0 tokens



Output tokens

model's reply



Overflow

Truncated

max

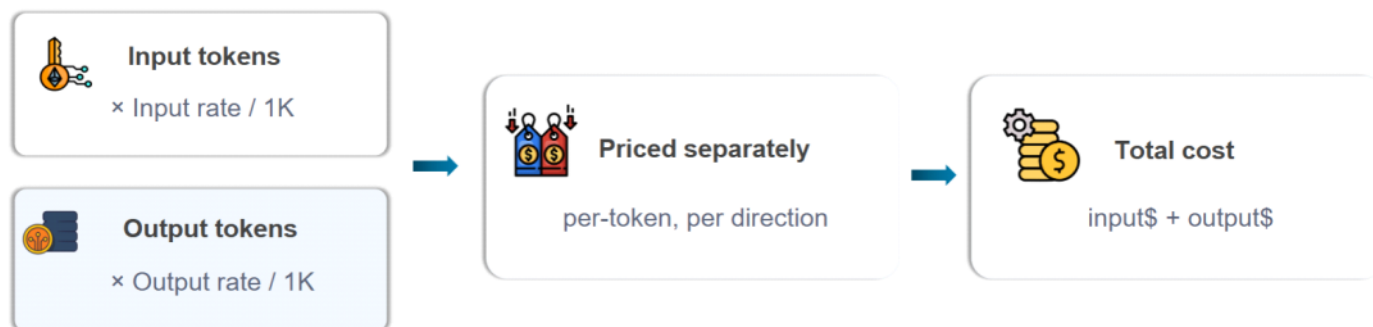
← beyond limit, dropped from view →



Longer prompts leave **less room for the answer**. Budgeting input vs. output is a real design decision.

Token Costs and Context Trade-Offs

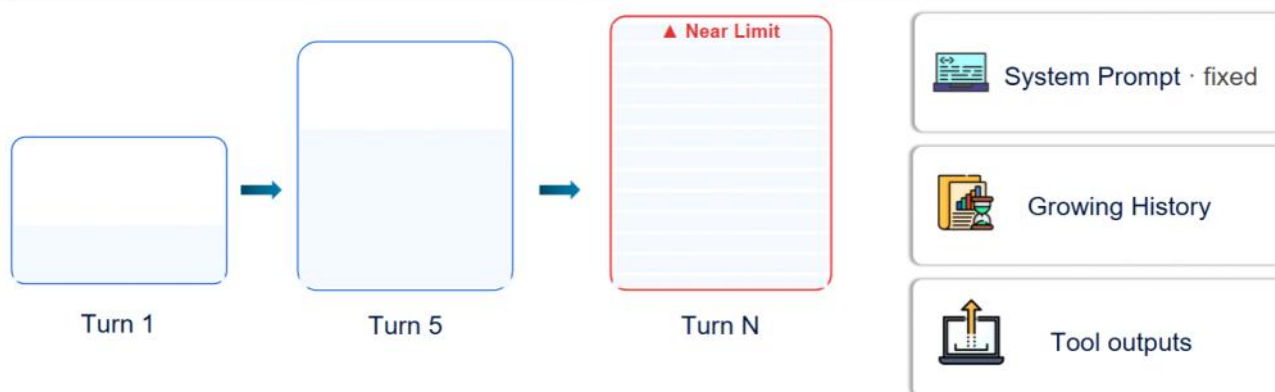
You pay **per token**. Input and output are metered at **different rates** and summed into the bill for each call.



Best Practice: output tokens usually cost more. Cap `max_tokens` and keep prompts lean, not just short answers.

Context Growth Across Agent Interactions

An agent loop keeps adding to the context. History and tool outputs accumulate until the window is **full**.



When the window becomes full, older or less important context may need to be removed, summarised, or retrieved again.

Context Management Strategies

The Context Management Problem

Every conversation has a fixed token budget. Once full, the oldest turns are dropped silently, not saved.

Context window — full

Context window(Fixed Size)

Turn 1

Dropped

Turn 2

Dropped

Turn 3

Kept

Turn 4

Kept

Turn 5

Kept

Turn 6

Current

Three Ways to Prevent Context Loss



Chunking

Splits large documents into pieces



Summarization

Preserves the gist, drops detail



Windowing

keeps only the last N turns

Chunking: Split Long Content Into Pieces

Break a long document into **fixed-size, overlapping chunks**, each processed independently.



Long Document
Too Big To Send



Split + Overlap
Overlapping Edges



Chunk 1



Chunk 2



Chunk 3



Embed Each
Index For Retrieval

Retrieval later pulls only the relevant chunks into context, not the entire document.

Summarisation: Compressing Older Context

Long-running conversations can quickly fill the context window. Summarisation keeps the important meaning while reducing the amount of text sent forward.

Detailed Conversation History

Summary Memory

Compact Context for the LLM

Purpose

Compress older messages and events
Preserve key facts, goals, and decisions

Benefits

Reduces token usage
Supports longer conversations

Limitation

Fine-grained details may be lost
Important facts must be retained carefully

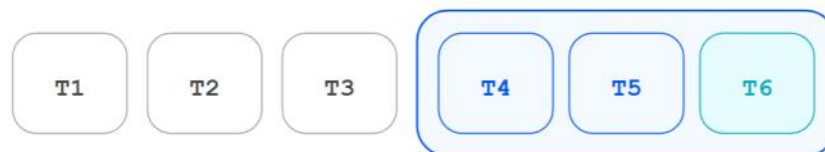
Windowing: Retaining the Most Recent Turns

A fixed-size window slides over the history; older turns drop out as new ones arrive.

window (N=3)



New Turn Arrives → Window Slides Right



Cheap and predictable, but it forgets: anything outside the window is gone. Best for short-horizon chat.

Choosing a Context Strategy

Match the **content** and **goal** to the strategy. Production systems often combine them.

What are you managing?

Large Static Corpus



Chunking + Retrieval

fetch only relevant pieces

Long Conversation



Summarization

must remember the gist

Only Recent Matters



Windowing

short-horizon chat

Conversational Memory

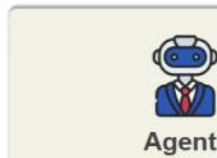
Conversational Memory Types

Different memory approaches store different kinds of context. The best option depends on conversation length, recall needs, and token budget.

Memory Type	What It Stores	Best Use
Buffer Memory	Recent messages kept in full	Short conversations that need exact recent context
Summary Memory	Compressed version of earlier conversation history	Long conversations where the gist matters more than every detail
Vector Memory	Past information retrieved by semantic similarity	Personalised recall and cross-session context

Persisting Conversational Memory

Checkpointers and stores help an agent remember what happened and use that memory later.



Reads the conversation and decides what to remember.



Memory

Examples: user name, previous message, approval status, user preference

The saved information helps the agent continue instead of starting from zero every time.



Simple Idea

Persistence means saving the agent's progress so the conversation can continue later.

Checkpointers and Stores



Checkpoint

Saves the current conversation state, like a bookmark in a workflow.

```
graph.invoke(  
  input,  
  config=config  
)
```

State is saved automatically.



Store

Saves useful facts for later, like a personal notebook for preferences.

```
store.put(  
  namespace,  
  key,  
  value  
)
```

Information is stored for later use.

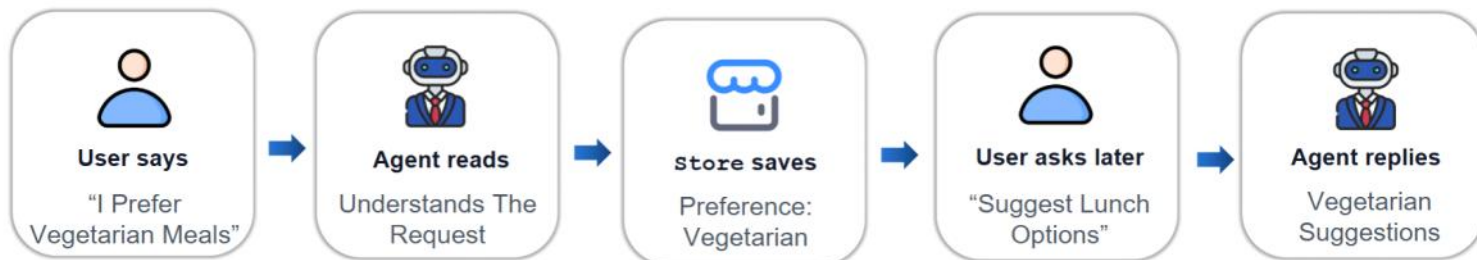


Checkpoint remembers where the current journey stopped.

Store remembers helpful information for future journeys.

Conversational Memory in Practice

The agent saves the conversation, remembers a preference, and uses it in the next answer.

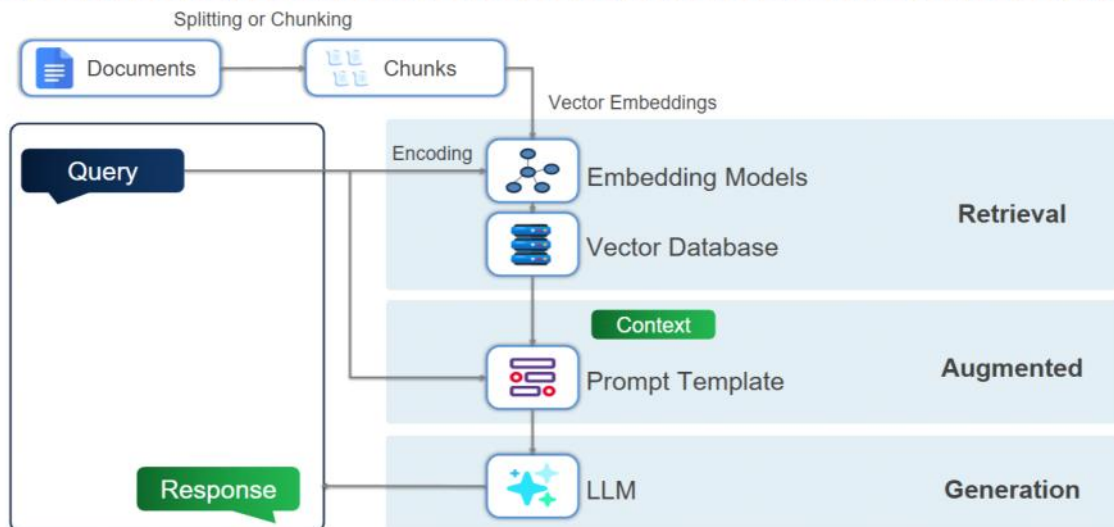


Demonstration 1: Building a Buffer and Summary Memory System

Retrieval as Context

Retrieval as Context for Grounded Responses

A query is used to retrieve relevant information from a knowledge base, which is then used to augment the prompt for LLM to generate a response.



Retrieved Context in the Generation Flow

How The Pipe Flows



```
The Complete RAG Chain

# --- indexing: runs once ---
docs = PyPDFLoader("policy.pdf").load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=800, chunk_overlap=120
).split_documents(docs)
store = Chroma.from_documents(chunks, embeddings)
retriever = store.as_retriever(search_kwargs={"k": 4})

# --- query: runs per question ---
prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY this context.\n"
    "Context: {context}\nQuestion: {question}"
)

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

chain.invoke("How long do refunds take?")
```

Managing Long Documents and Multi-Turn Conversation History

Document Loaders for Context Ingestion

Document loaders import content from different sources into a standardized document format so it can be processed by language models and other pipeline components

Types of Document Loaders

Integrated Sources

Custom Loaders

CSVLoader

PyPDFLoader

WebBaseLoader

Demonstration 2 : Grounding LLM Responses with Retrieved Context

The Document Object and Custom Loaders

Anatomy of a Document

`.page_content`

The raw text the model will actually read.

```
"Refunds are processed in 5-7 business days."
```

`.metadata`

Where it came from, this is what lets you cite sources.

```
"source"      : "policy.pdf"
"page"        : 3
"section"     : "Returns"
```

```
Writing Your Own Loader

from langchain_core.document_loaders import BaseLoader

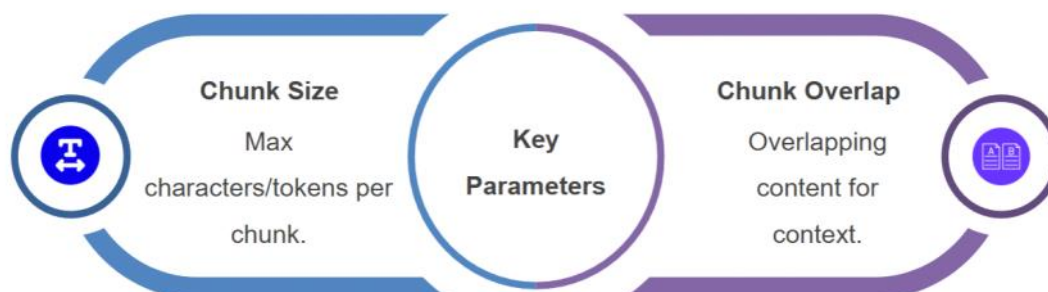
class TicketLoader(BaseLoader):
    def lazy_load(self):
        for row in self.rows:
            yield Document(
                page_content=row["body"],
                metadata={"id": row["id"]},
            )
```

Use `lazy_load()` : It yields one Document at a time, so a 10 GB corpus never lands in memory.

Text Splitting for Long Documents

Text splitters divide long documents into smaller, manageable chunks for processing and retrieval.

This is useful when documents exceed context limits or need to be indexed for retrieval



Recursive and Semantic Chunking

Same document, two cutting strategies: one follows structure, one follows meaning.

`RecursiveCharacterTextSplitter`

Cuts on structure:
paragraph → line → word



Equal sizes · fast · no model needed

`SemanticChunker`

Cuts where the topic shifts:
keeps related ideas together.



Uneven sizes · one coherent idea per chunk

Maintaining Multi-turn Conversation History

When a user comes back later, the agent needs the right memory.



`thread_id`

Keeps one conversation session
separate from another.

`user_id + store`

Keeps useful user facts across many
sessions.

Thread-Level and User-Level Memory

Conversation history must be saved with the correct identity.

Multi-turn conversation

A chat where one answer depends on earlier messages.

thread_id

The ID for one running conversation or session.

store

A longer-term place to save reusable facts and preferences.

Example



thread_id
= one notebook page



checkerpoint
= saves that page



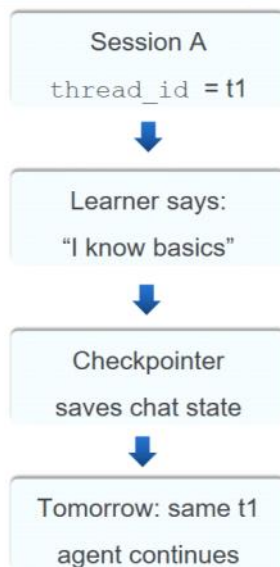
user_id/store
= user profile file



Personalized Reply

Continuing Context Across Sessions

Example: Continue learning across sessions



```
Conversation History

from langgraph.checkpoint.memory import MemorySaver
# Create a checkpoint
memory = MemorySaver()
# Compile the graph with memory
graph = builder.compile(checkpointer=memory)

# Session A
config = {
    "configurable": {
        "thread_id": "t1"
    }
}

graph.invoke(
    {"messages": "I know Python basics"},
    config=config
)

# Same session later...
graph.invoke(
    {"messages": "Continue my revision plan"},
    config=config
)
```

Demonstration 3 : Chunking and Summarizing Long Documents for Context

Measuring and Optimizing Token Usage

Measuring Token Usage with tiktoken

Measure the **real token count** before sending. It is almost never the word count.



Never estimate cost or window fit by word count. Use `encode()` and `len()` for the truth.

Context Optimization

Context optimization focuses on providing the **right amount and structure of information** to enable high-quality, maintainable, and correct code generation.



Balancing Context Depth

Insufficient context

Missing details and weak grounding

Excessive context

More noise, cost, and distraction



Focused context

Only task relevant information

Prioritized context

Critical information is emphasized

Techniques for Context Optimization



Context ordering

01



Scope limitation

02



Irrelevant detail removal

03



Explicit prioritization

06



Context segmentation

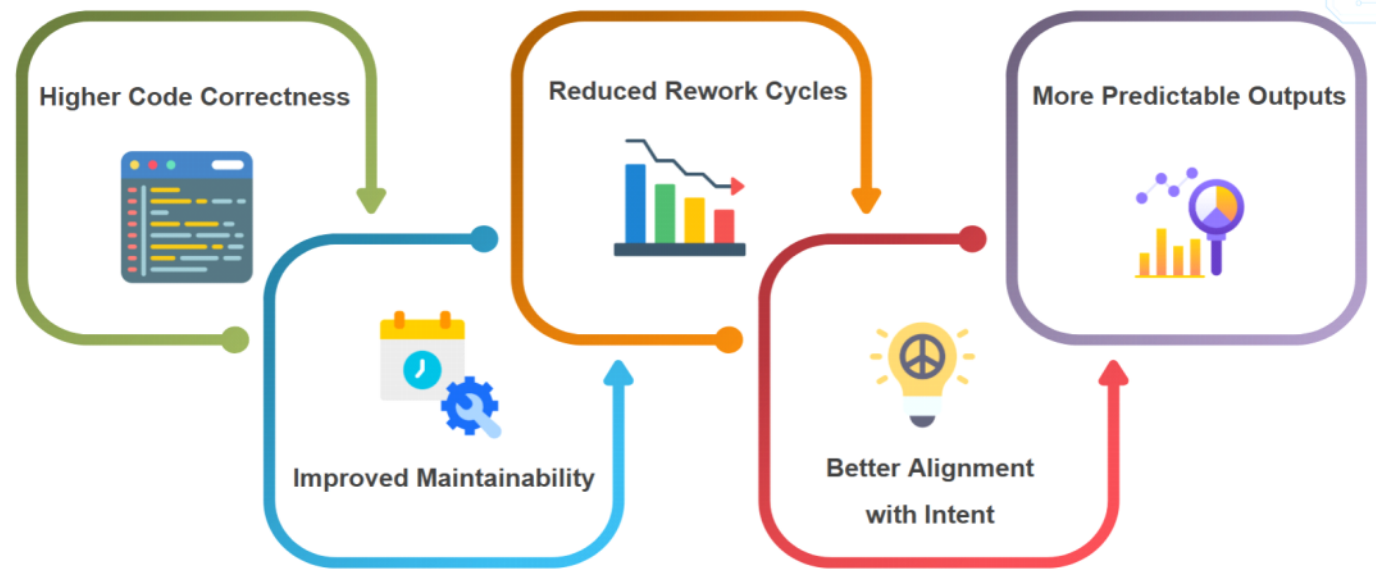
05



Progressive disclosure

04

Outcomes of Optimized Context



Demonstration 4 : Measuring and Optimizing Token Usage in Conversations

Summary

In this lesson, you have learned to:

- e! Explain context engineering for AI applications and agents.
- e! Understand context-window limits, costs, and trade-offs.
- e! Apply chunking, summarisation, and windowing strategies.
- e! Compare buffer, summary, and vector memory.
- e! Use retrieval to ground LLM responses.
- e! Manage long documents and multi-turn conversation history.
- e! Measure and optimise token usage.